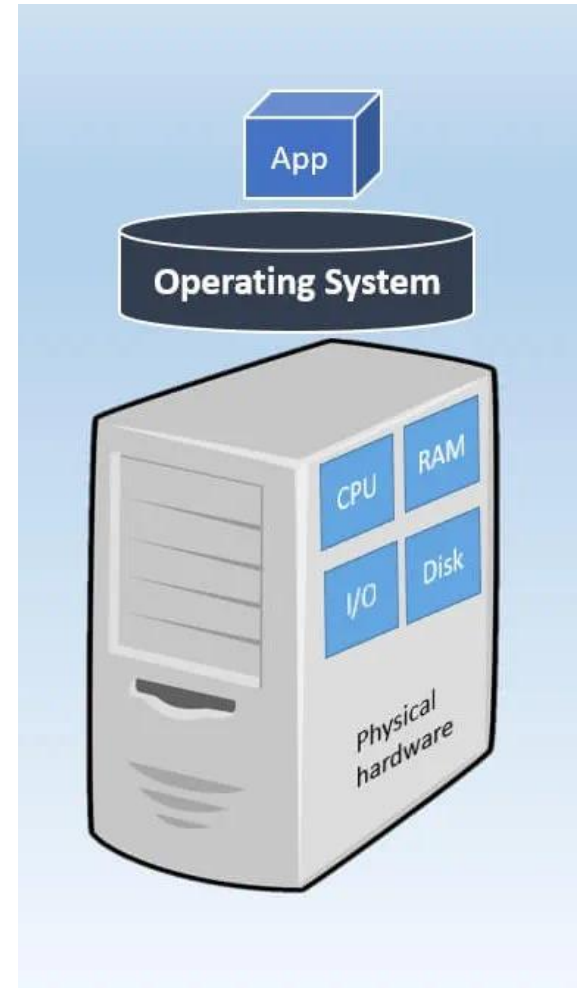


Containerization

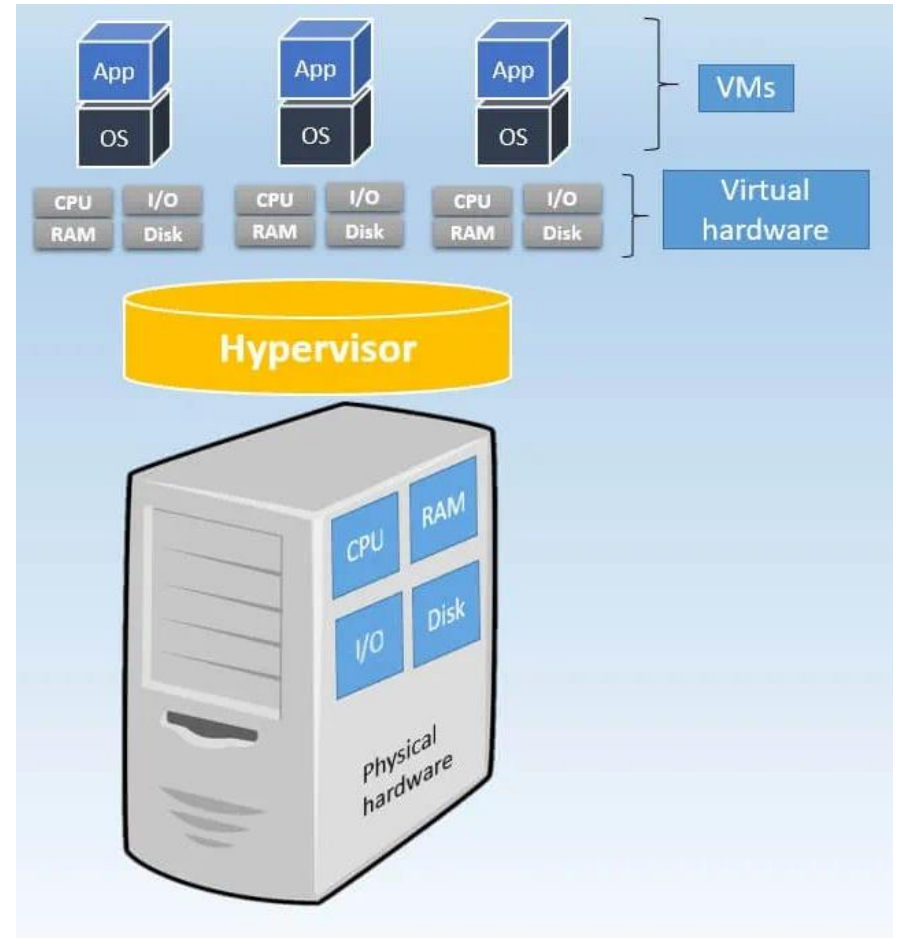
The Start – Physical Server

- A physical server (or bare-metal server) is a standalone, tangible hardware unit containing its own CPU, RAM, storage, and network interfaces, which runs its own dedicated operating system.
- Problems:
 - Cost
 - Scalability Limitations
 - Maintenance Burden
 - Slow Provisioning
 - Wasted Resources



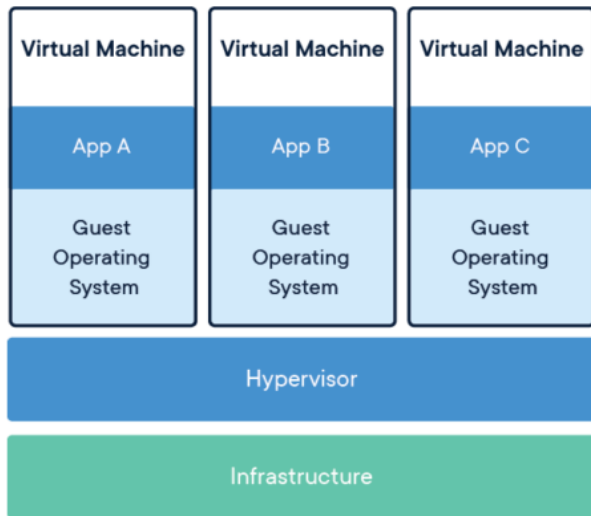
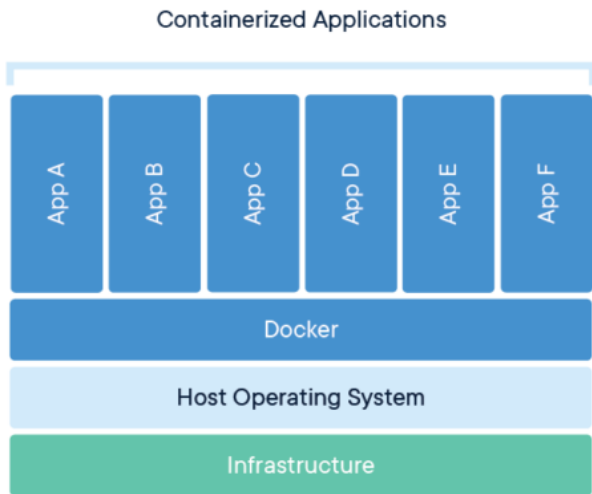
The Start – Virtual Machine

- A virtual machine (VM) is a software computer used as an emulation of an actual physical computer.
- More cost-effective option than a physical server.
- Problems:
 - VM tax

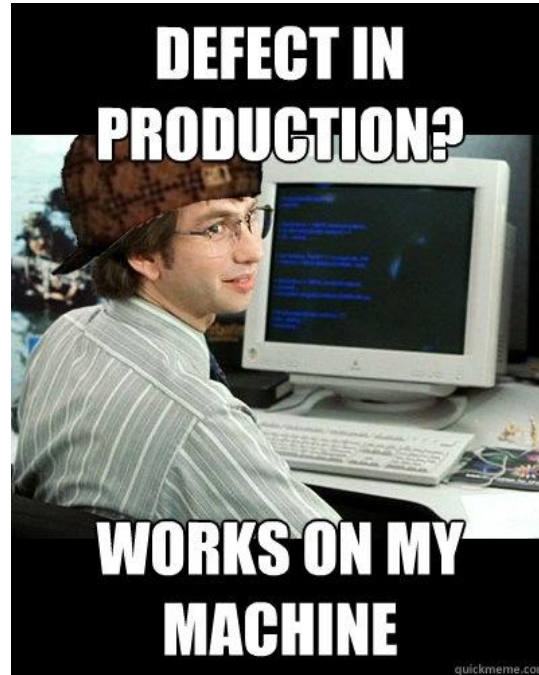


The Start – Container

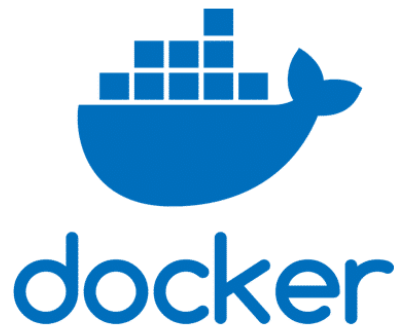
- Containers may be described as packages that include everything needed to run a single application or microservice.
- Containers allow the application to be run quickly and reliably from anywhere, right from desktop computers to physical and virtual servers, and even the cloud.



Why Containerization is important?



Tools used in Containerization



Container Orchestration

(run & manage containers at scale)

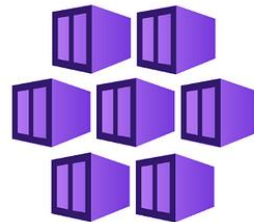
1. Kubernetes
2. AWS ECS (Elastic Container Service)
3. Azure Container Service



AWS ECS

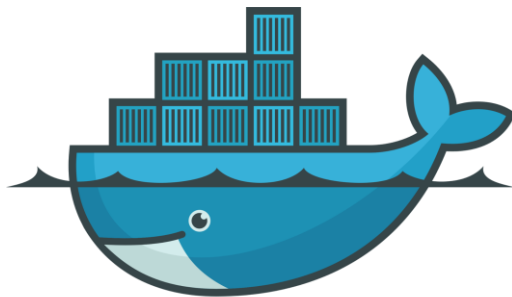


kubernetes



Azure Kubernetes Service (AKS)

Docker



What is Docker?

Docker is a platform that enables developers to automate the deployment of applications within lightweight, portable containers.



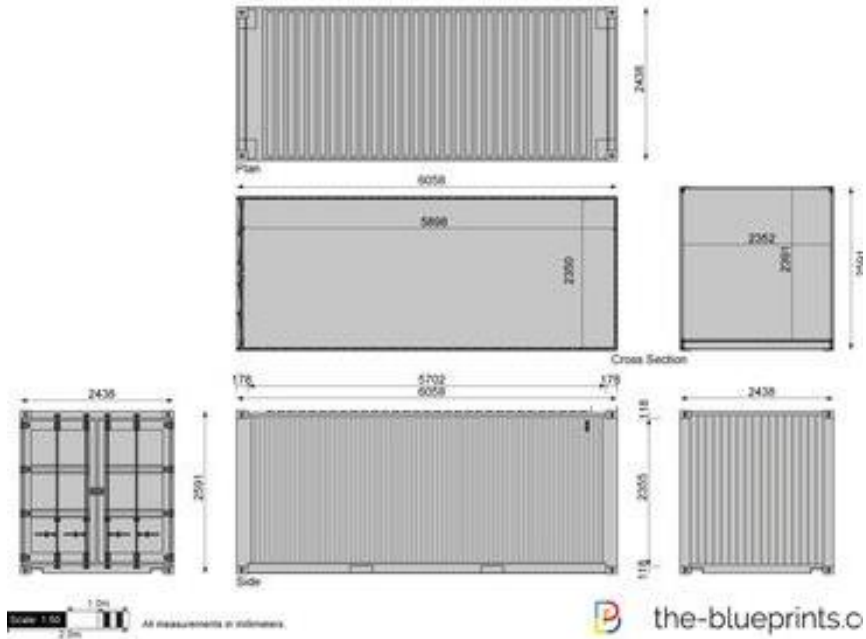
What does Docker mean?

A person employed in a port to load and unload ships.



What does Docker mean?

Standard ISO 20' 20-foot Container



Docker Components - Image

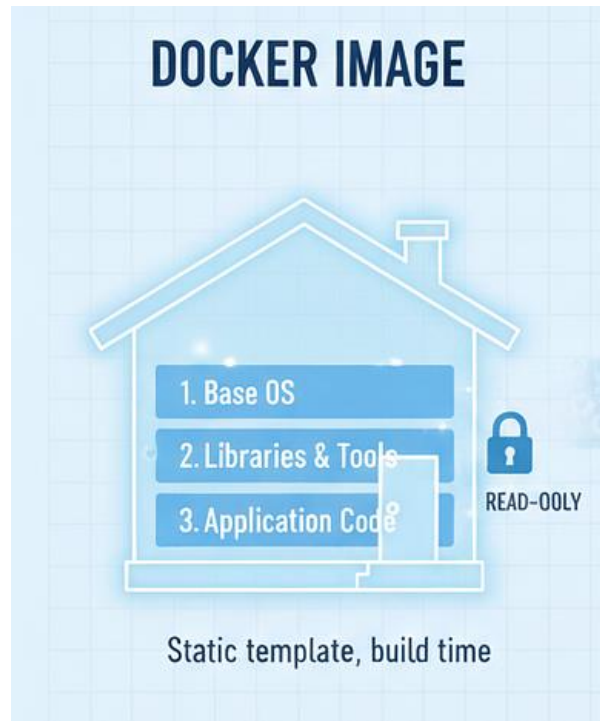
A **Docker image** is a lightweight, standalone, executable package that includes everything needed to run a piece of software.

It includes the code, runtime, system tools, libraries, and settings.

It serves as a read-only blueprint or template for creating Docker containers.

Key Characteristics

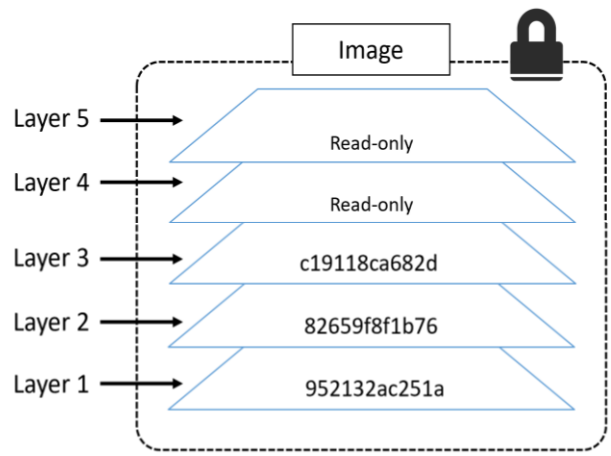
- Immutability
- Portability
- Layered Architecture



Docker Components - Image

Images are composed of a series of read-only layers.

```
$ docker image pull ubuntu:latest
latest: Pulling from library/ubuntu
952132ac251a: Pull complete
82659f8f1b76: Pull complete
c19118ca682d: Pull complete
8296858250fe: Pull complete
24e0251a0e2c: Pull complete
Digest: sha256:f4691c96e6bbaa99d...28ae95a60369c506dd6e6f6ab
Status: Downloaded newer image for ubuntu:latest
docker.io/ubuntu:latest
```



Docker Components - Image

Multiple images can, and do, share layers. This leads to efficiencies in space and performance.

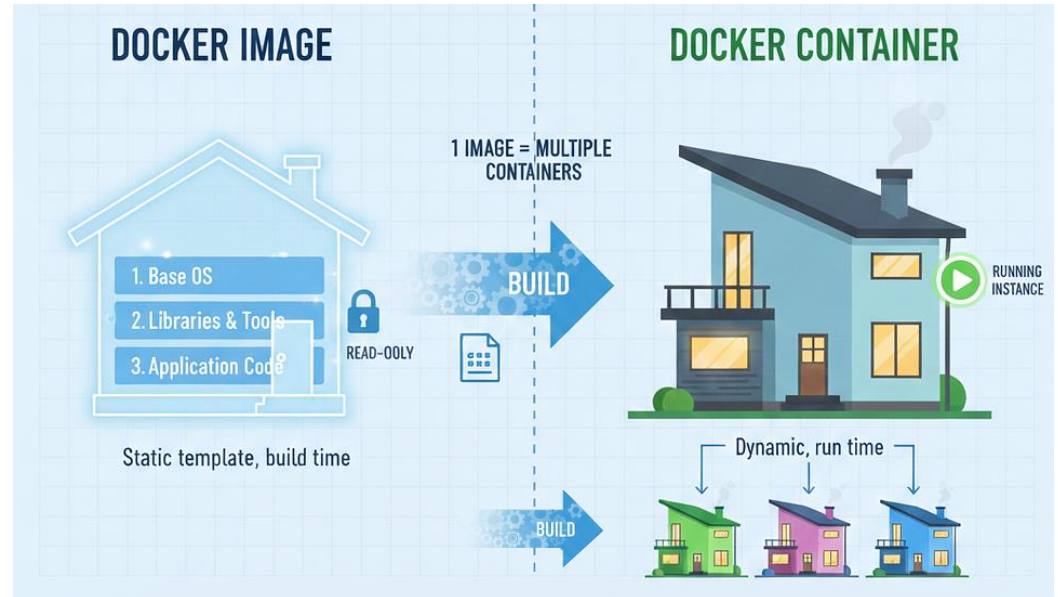
```
$ docker image pull -a nigelpoulton/tu-demo
latest: Pulling from nigelpoulton/tu-demo
aad63a933944: Pull complete
f229563217f5: Pull complete
<Snip>>
Digest: sha256:c9f8e18822...6cbb9a74cf

v1: Pulling from nigelpoulton/tu-demo
aad63a933944: Already exists
f229563217f5: Already exists
```

Docker Components - Container

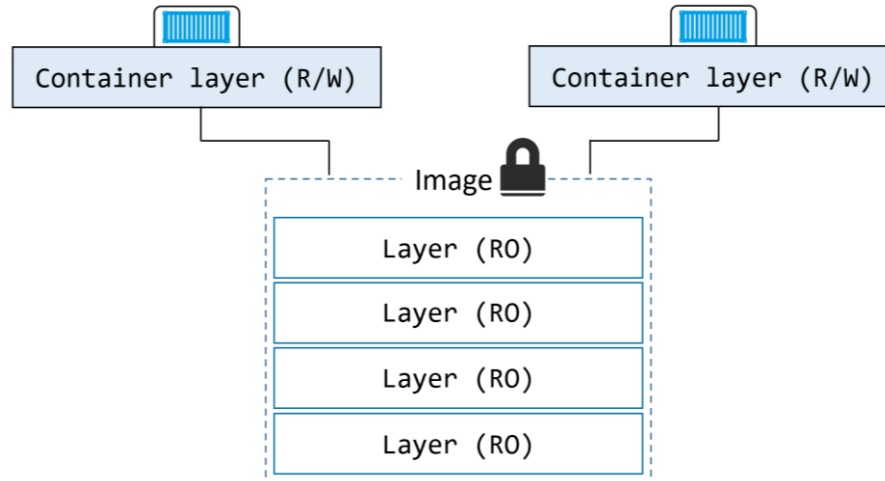
A **Docker container** is a live, runnable instance of an image with an additional writable layer on top for runtime changes.

Multiple containers can be created from the same image.

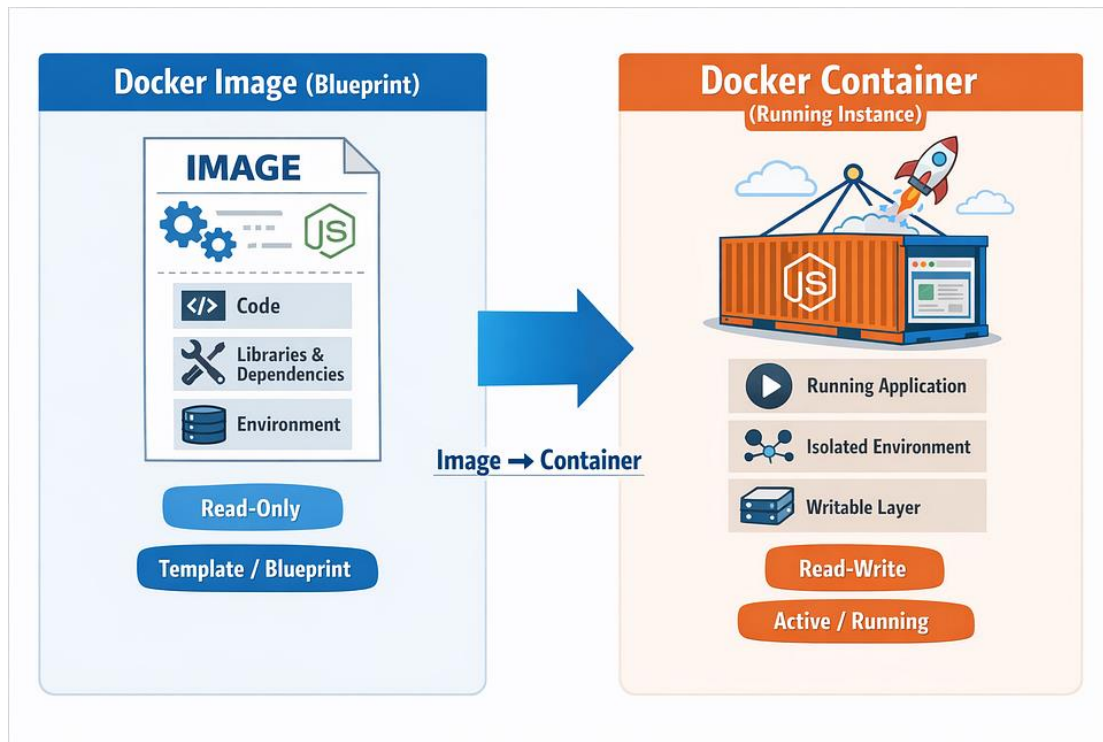


Docker Components - Container

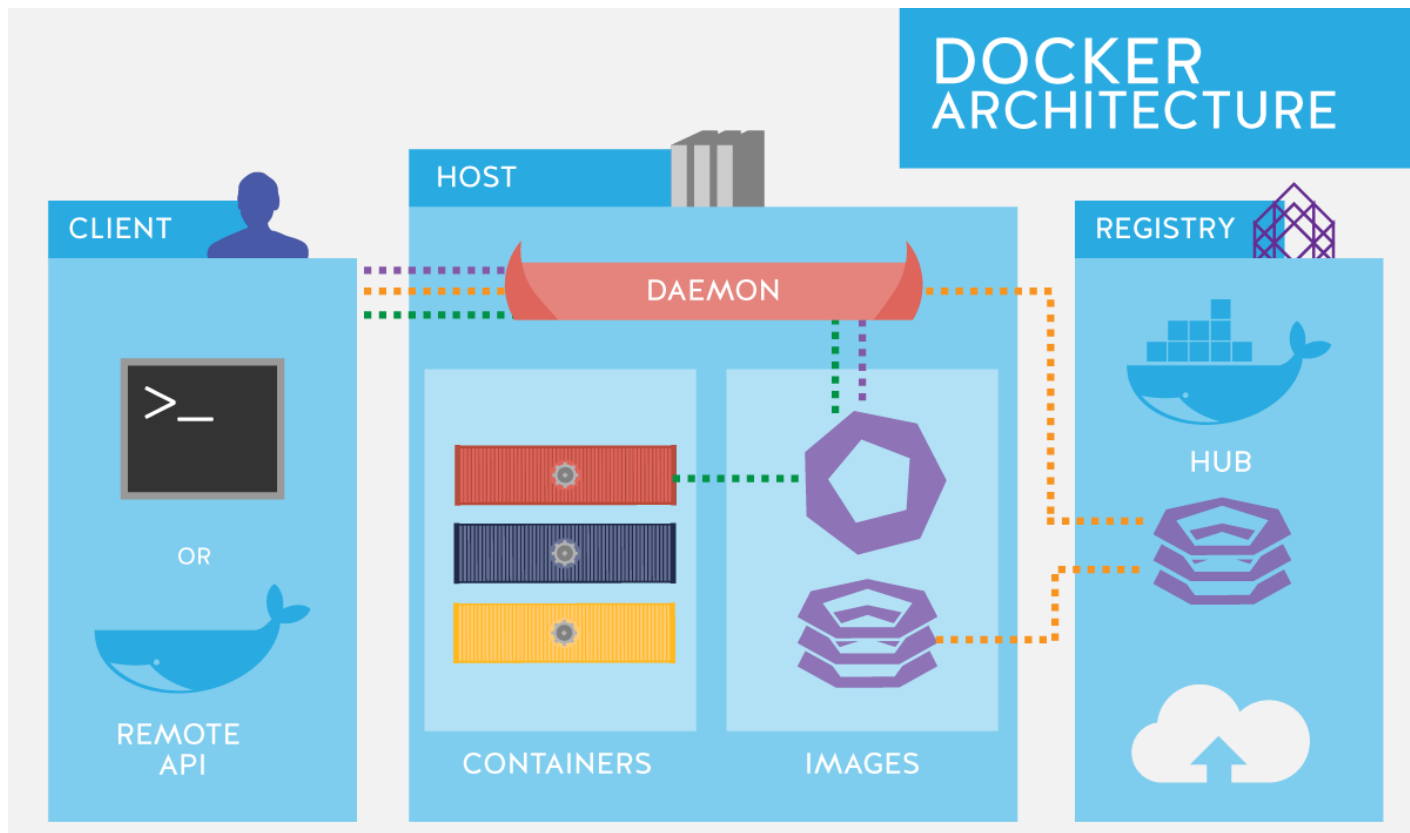
- Every Docker container is created by adding a thin read-write layer on top of the read-only image it's based on
- It gets created when the container is created and it gets deleted when the container is deleted



Images vs Containers



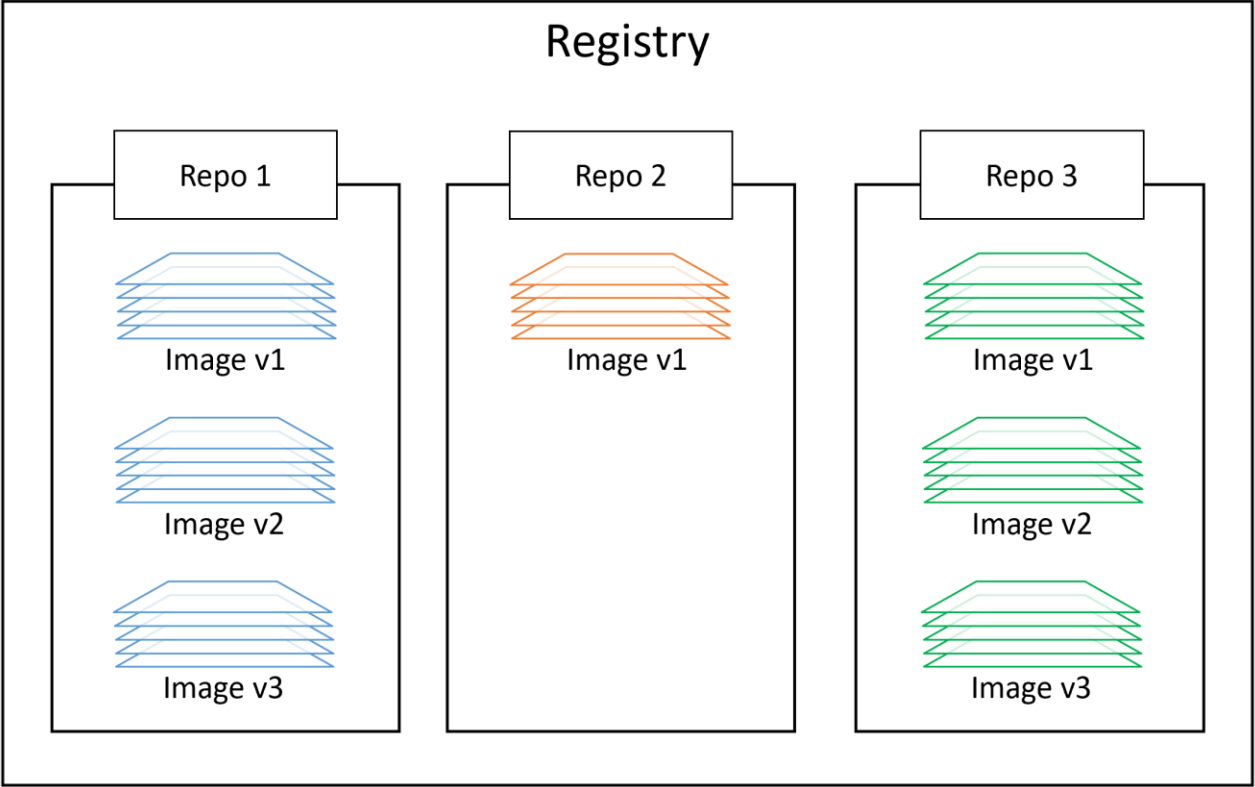
Docker Architecture Overview



Docker Registry



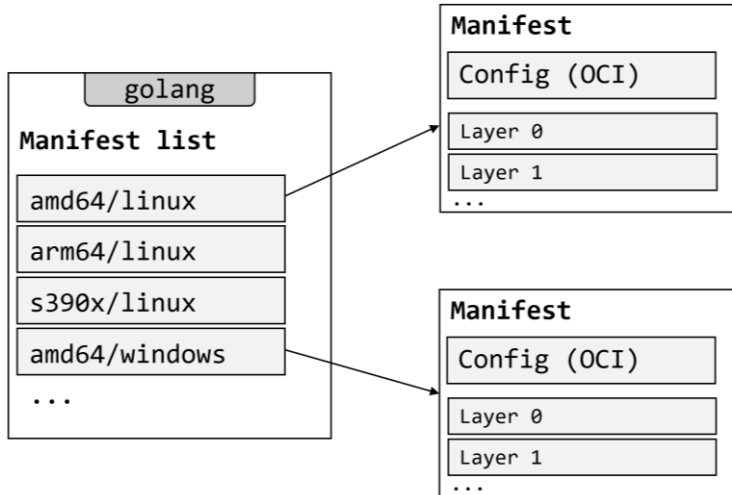
Docker Registry



Docker Components - Image

Multi-architecture images

- A Docker manifest is a JSON-formatted file that tells the Docker engine what components are needed to build and run a container from that image
- A single image tag supporting multiple platforms and architectures.



Docker Components - Image

By default, Docker automatically selects the correct image by matching the **host platform**:

- The Docker daemon knows the host OS and CPU architecture

Manual Override with `--platform`

```
# Pull a specific platform
docker pull --platform linux/arm64 nginx

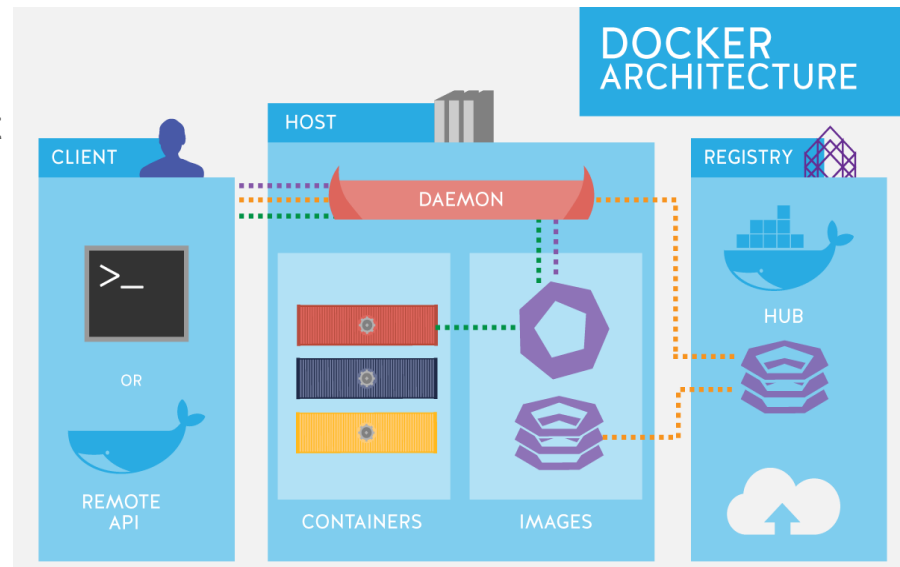
# Run a specific platform (with emulation via QEMU)
docker run --platform linux/arm64 nginx

# Build targeting a specific platform
docker build --platform linux/amd64,linux/arm64 .
```

Docker Architecture Overview

Docker uses a client-server architecture:

1. **Docker Client:** the primary way users interact with Docker via type Docker commands (send requests)
2. **Docker Daemon:** manages Docker objects like images, containers, networks, and volumes. (server that handle requests)
3. **Docker Registry:** stores Docker images (e.g., Docker Hub).



TAKE A BREAK!



Basic Docker Commands



Example 1

Running your first container

Let's start by running a simple container:

```
docker run hello-world
```

This command does the following:

1. Checks for the hello-world image locally
2. If not found, pulls the image from Docker Hub
3. Creates a container from the image
4. Runs the container, which prints a hello message
5. Exits the container

Running your first container

Let's start by running a simple container:

```
docker run hello-world
```

-it: run in interactive mode and specify a terminal

--name: specify the container name

-d: run in detach mood

Basic Docker Commands

Here are some essential Docker commands for working with containers:

Listing Containers

To see all running containers:

```
docker ps
```

To see all containers (including stopped ones):

```
docker ps -a
```

Basic Docker Commands

Starting and Stopping Containers:

To stop a running container:

```
docker stop <container_id_or_name>
```

To start a stopped container:

```
docker start <container_id_or_name>
```

To restart a container:

```
docker restart <container_id_or_name>
```

Basic Docker Commands

Removing Containers:

To remove a stopped container:

```
docker rm <container_id_or_name>
```

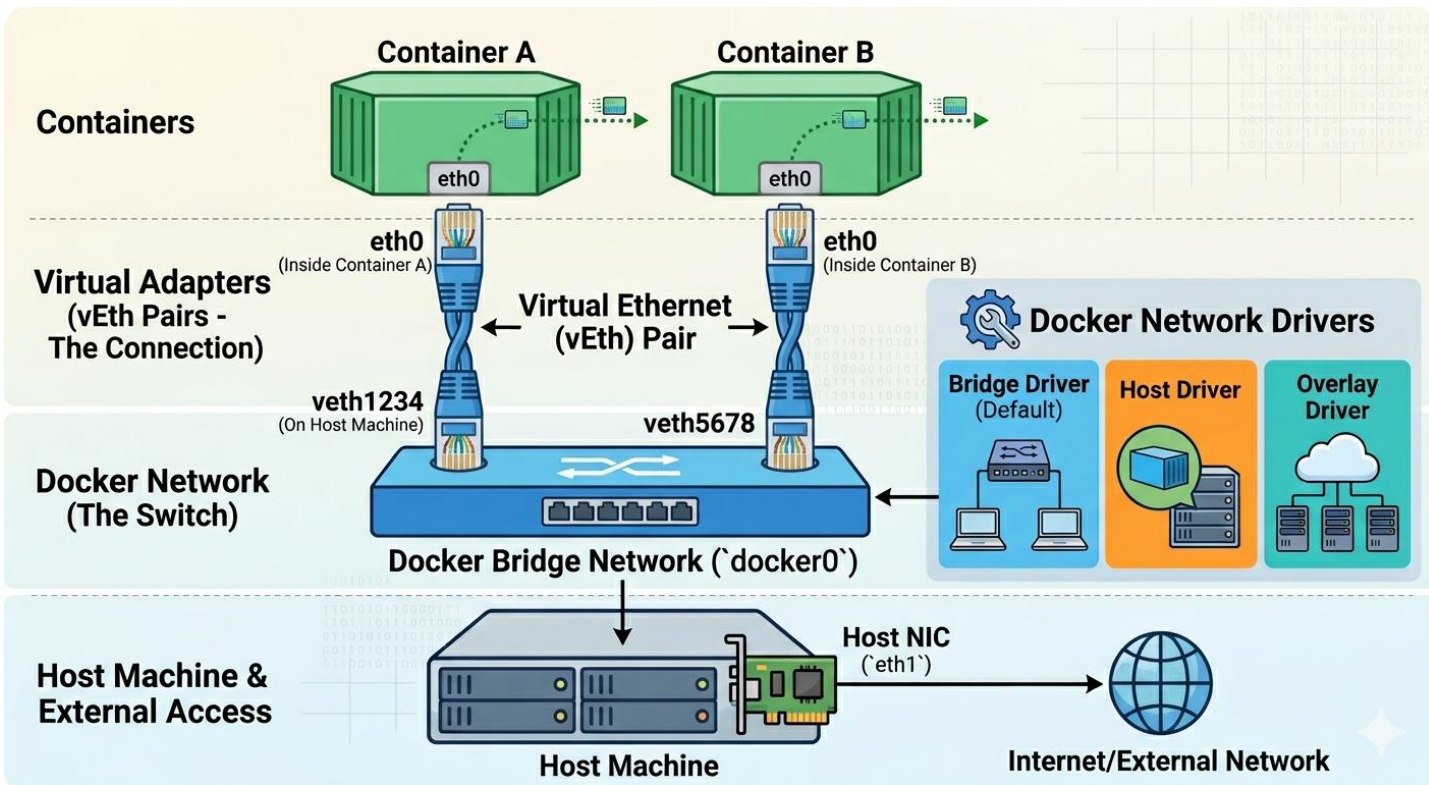
To force remove a running container:

```
docker rm -f <container_id_or_name>
```

Docker in VSCode

Let's go to vsCode

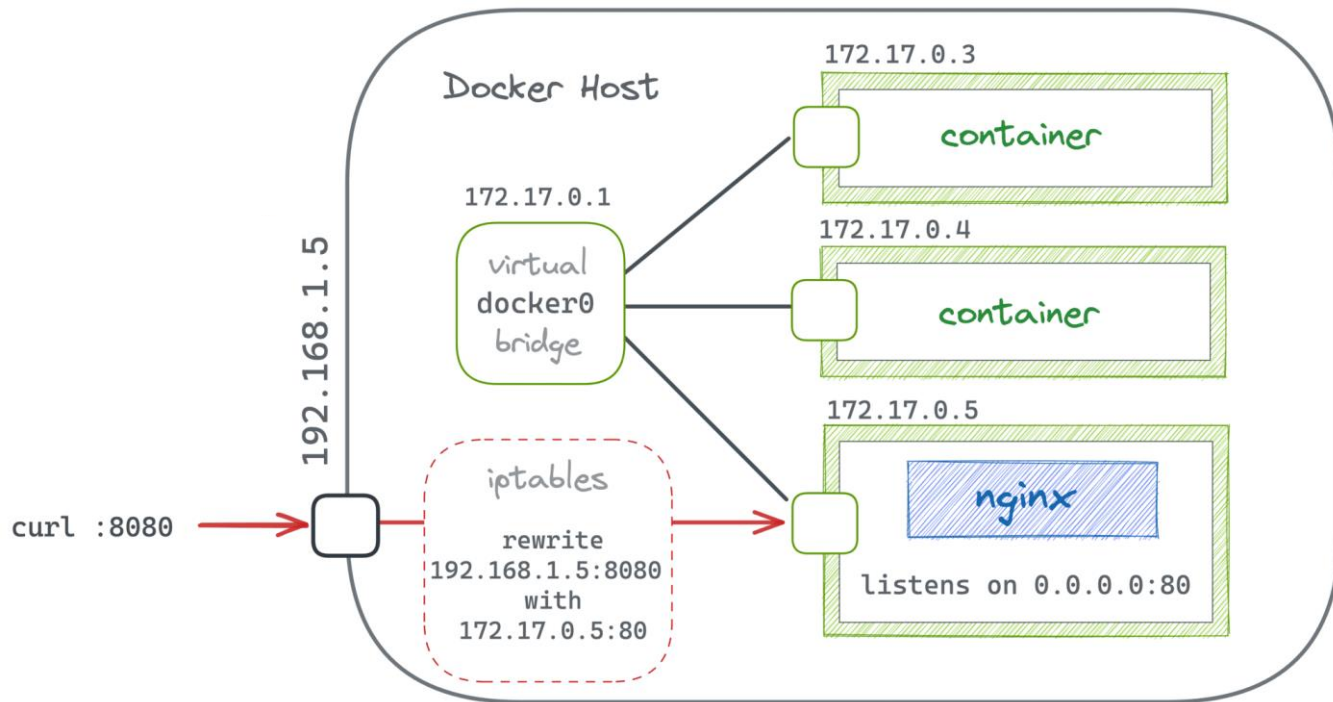
Docker Network



Basic Docker Commands

Port Mapping:

```
docker run -d -p 8080:80 hello-world
```





Example 2

Basic Docker Commands

Saving Container Image:

```
docker commit <container-name> <user-name>/<repo-name>:<tag>
```

Basic Docker Commands

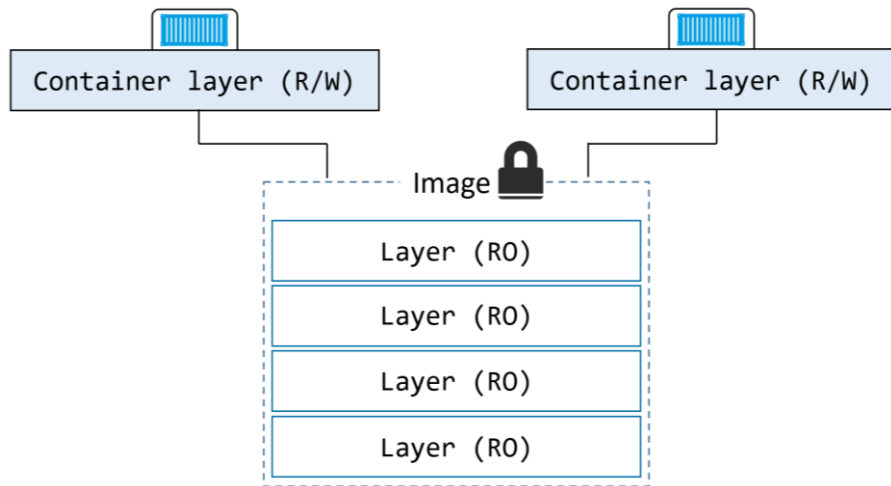
Container Logs:

To show the last 100 lines of container logs and keep showing any new logs:

```
docker log --tail 100 -f hello-world-container
```

Docker Storage

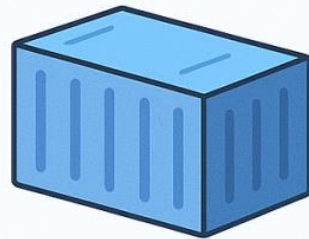
- Every Docker container is created by adding a thin read-write layer on top of the read-only image it's based on
- It gets created when the container is created and it gets deleted when the container is deleted



Docker Storage

- A **bind mount** in Docker is a storage mechanism that maps a specific file or directory from your host machine's filesystem directly into a container.

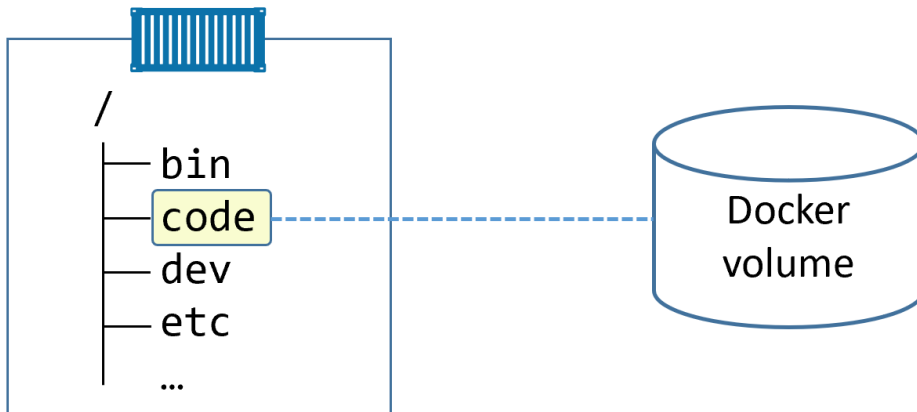
```
docker run -d -v /home/user/app:/app hello-world
```



BIND MOUNT

Docker Storage

- *Volumes* are the recommended way to persist data in containers.
 - Volumes are independent objects that are not tied to the lifecycle of a container
 - Volumes can be mapped to specialized external storage systems
 - Volumes enable multiple containers on different Docker hosts to access and share the same data

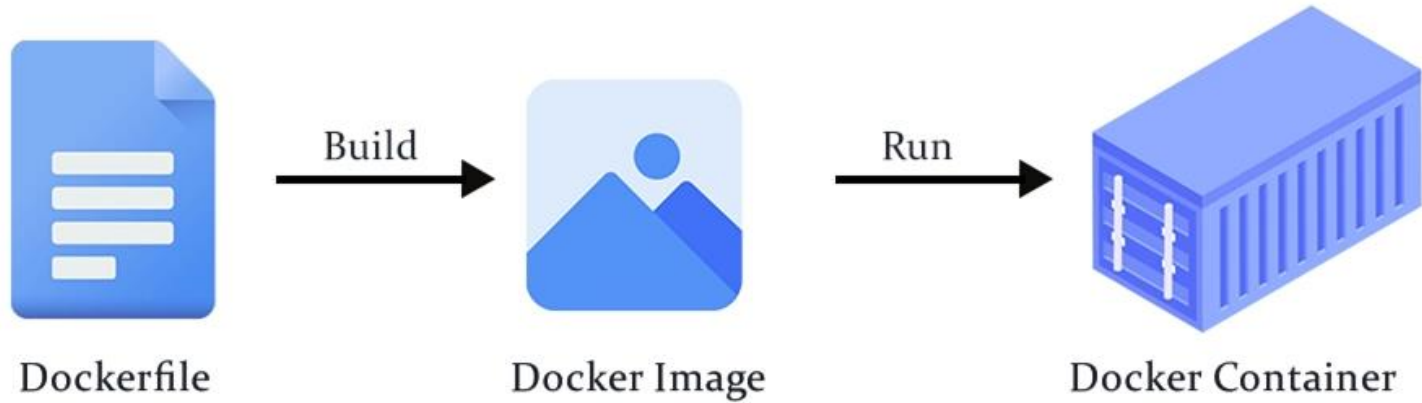




Example 3

Containerizing an Application

Containerizing an Application



DockerFile

Dockerfile — a document that contains a series of ordered instructions used to create a Docker image automatically.

FROM `python:latest`

- Pull the latest official Python image from Docker Hub.

WORKDIR `/app`

- Set the working directory inside the container to `/app`.

Note: it creates the directory if it doesn't exist

COPY `requirements.txt .`

- Copy the `requirements.txt` file from the build context to the working directory (`/app`) inside the container.

RUN `pip install -r requirements.txt`

- Install Python dependencies listed in `requirements.txt`.

ARG `BATCH_SIZE`

- Create a variable that lives within the dockerfile scope only

ENV `BATCH_SIZE=${BATCH_SIZE}`

- Create an environment variable

COPY `hello.py .`

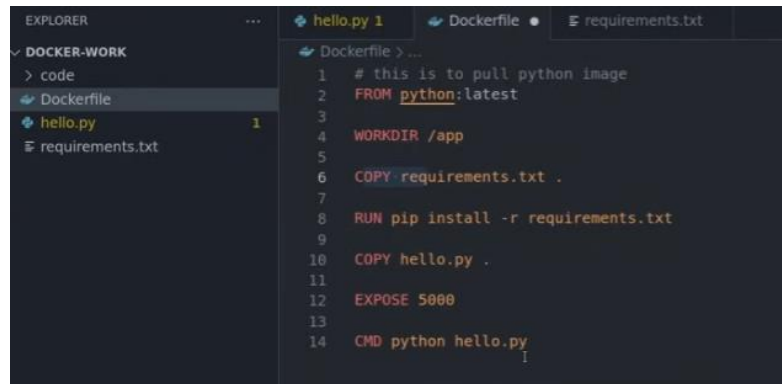
- Copy the `hello.py` script into the container working directory.

EXPOSE `5000`

- Inform Docker that the application will use port 5000 (useful for web apps, like Flask).

CMD `python hello.py`

- This is the command that will run when the container starts: it runs your `hello.py` script



The screenshot shows a code editor with two panels. The left panel, titled 'EXPLORER', shows a file tree for a project named 'DOCKER-WORK'. It contains a 'code' directory, a 'Dockerfile', and two files: 'hello.py' and 'requirements.txt'. The 'Dockerfile' is selected. The right panel shows the content of the 'Dockerfile' with line numbers 1 through 14. The instructions are: 1. A comment '# this is to pull python image', 2. 'FROM python:latest', 3. An empty line, 4. 'WORKDIR /app', 5. An empty line, 6. 'COPY requirements.txt .', 7. An empty line, 8. 'RUN pip install -r requirements.txt', 9. An empty line, 10. 'COPY hello.py .', 11. An empty line, 12. 'EXPOSE 5000', 13. An empty line, 14. 'CMD python hello.py'.

```
docker build -t <user-name>/<repo-name>:<tag> .
```

DockerFile

```
docker build \  
  --build-arg APP_VERSION=2.1.0 \  
  --build-arg BUILD_USER=admin \  
  -t my-app:latest .
```

```
docker run --env-file config.env my-app-image
```

```
docker build --secret id=my_secret_key,src=path/to/secret.txt -t my-image .
```

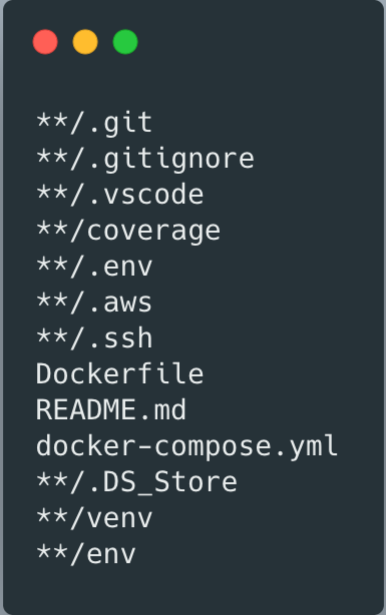
DockerFile

```
# Push cache to registry
docker build \
  --cache-to=type=registry,ref=myrepo/myapp:cache,mode=max \
  --cache-from=type=registry,ref=myrepo/myapp:cache \
  -t myrepo/myapp:latest .

# Use local directory cache
docker build \
  --cache-to=type=local,dest=/tmp/docker-cache,mode=max \
  --cache-from=type=local,src=/tmp/docker-cache \
  .
```

DockerFile

.dockerignore — a plain text file used to **exclude files and directories** from the Docker build context



```
**/.git
**/.gitignore
**/.vscode
**/coverage
**/.env
**/.aws
**/.ssh
Dockerfile
README.md
docker-compose.yml
**/.DS_Store
**/venv
**/env
```



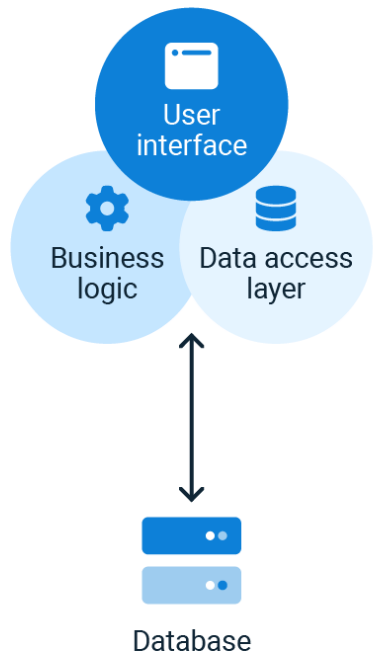
Example 4

TAKE A BREAK!

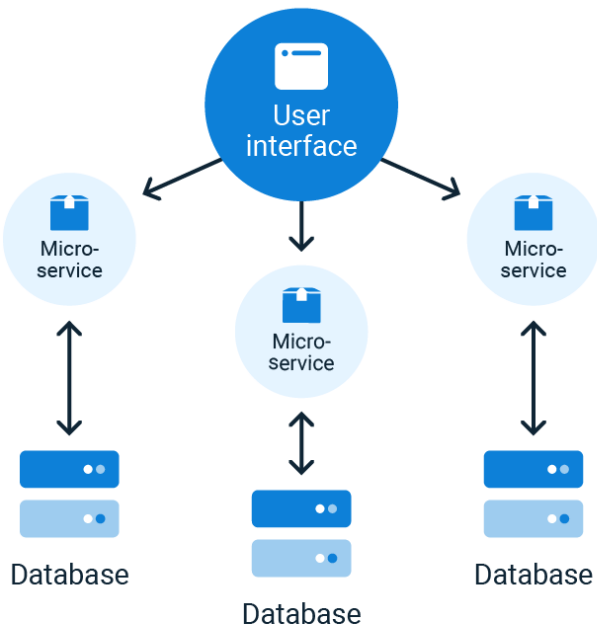


Docker Compose

Monolithic Architecture



Microservice Architecture

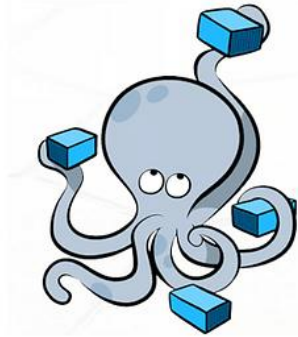


Docker Compose

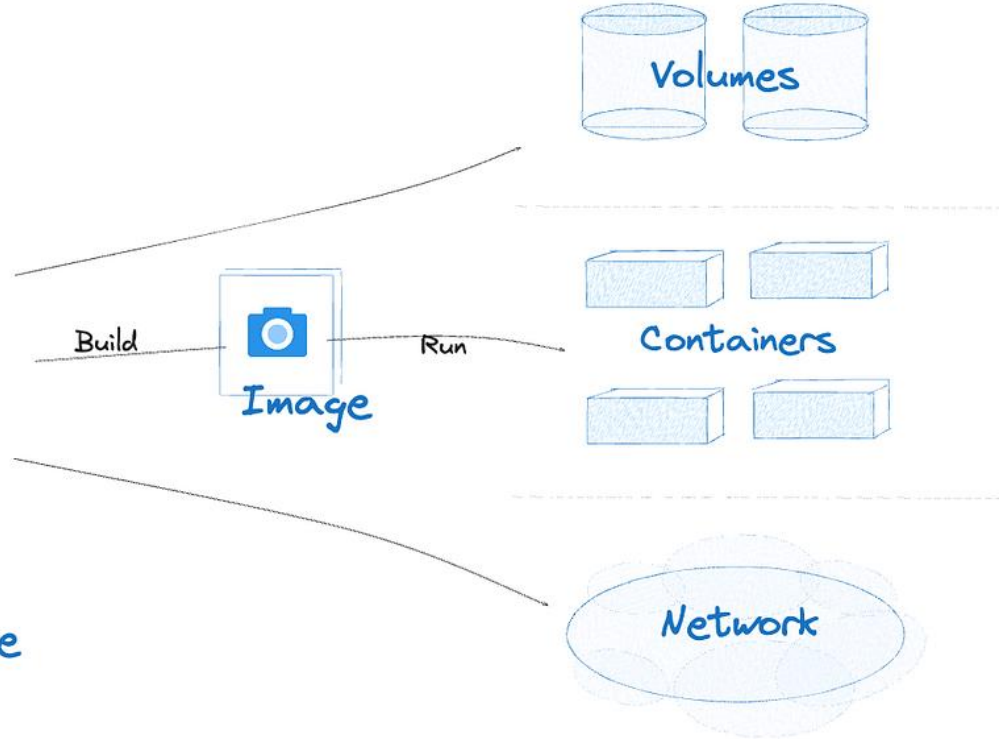
```
docker-compose.yml

services:
  app:
    image: node:18-alpine
    command: sh -c 'yarn install && yarn run dev'
    ports:
      - 127.0.0.1:3000:3000
    working_dir: /app
    volumes:
      - ./app
    environment:
      MYSQL_HOST: mysql
      MYSQL_USER: root
      MYSQL_PASSWORD: secret
      MYSQL_DB: todos
  mysql:
    image: mysql:8.0
    volumes:
      - todo-mysql-data:/var/lib/mysql
    environment:
      MYSQL_ROOT_PASSWORD: secret
      MYSQL_DATABASE: todos
volumes:
  todo-mysql-data:
```

Config YAML



Docker Compose



Docker file vs compose

- A **Dockerfile** is a script for building a single container image, while **Docker Compose** is a tool for running and orchestrating one or more containers as a unified application.

A screenshot of a code editor showing a Dockerfile. The editor has tabs for 'hello.py 1', 'Dockerfile', and 'requirements.txt'. The Dockerfile content is as follows:

```
1 # this is to pull python image
2 FROM python:latest
3
4 WORKDIR /app
5
6 COPY requirements.txt .
7
8 RUN pip install -r requirements.txt
9
10 COPY hello.py .
11
12 EXPOSE 5000
13
14 CMD python hello.py
```

A screenshot of a docker-compose.yml file. The content is as follows:

```
docker-compose.yml

services:
  app:
    image: node:18-alpine
    command: sh -c 'yarn install && yarn run dev'
    ports:
      - 127.0.0.1:3000:3000
    working_dir: /app
    volumes:
      - ./app
    environment:
      MYSQL_HOST: mysql
      MYSQL_USER: root
      MYSQL_PASSWORD: secret
      MYSQL_DB: todos

  mysql:
    image: mysql:8.0
    volumes:
      - todo-mysql-data:/var/lib/mysql
    environment:
      MYSQL_ROOT_PASSWORD: secret
      MYSQL_DATABASE: todos

volumes:
  todo-mysql-data:
```

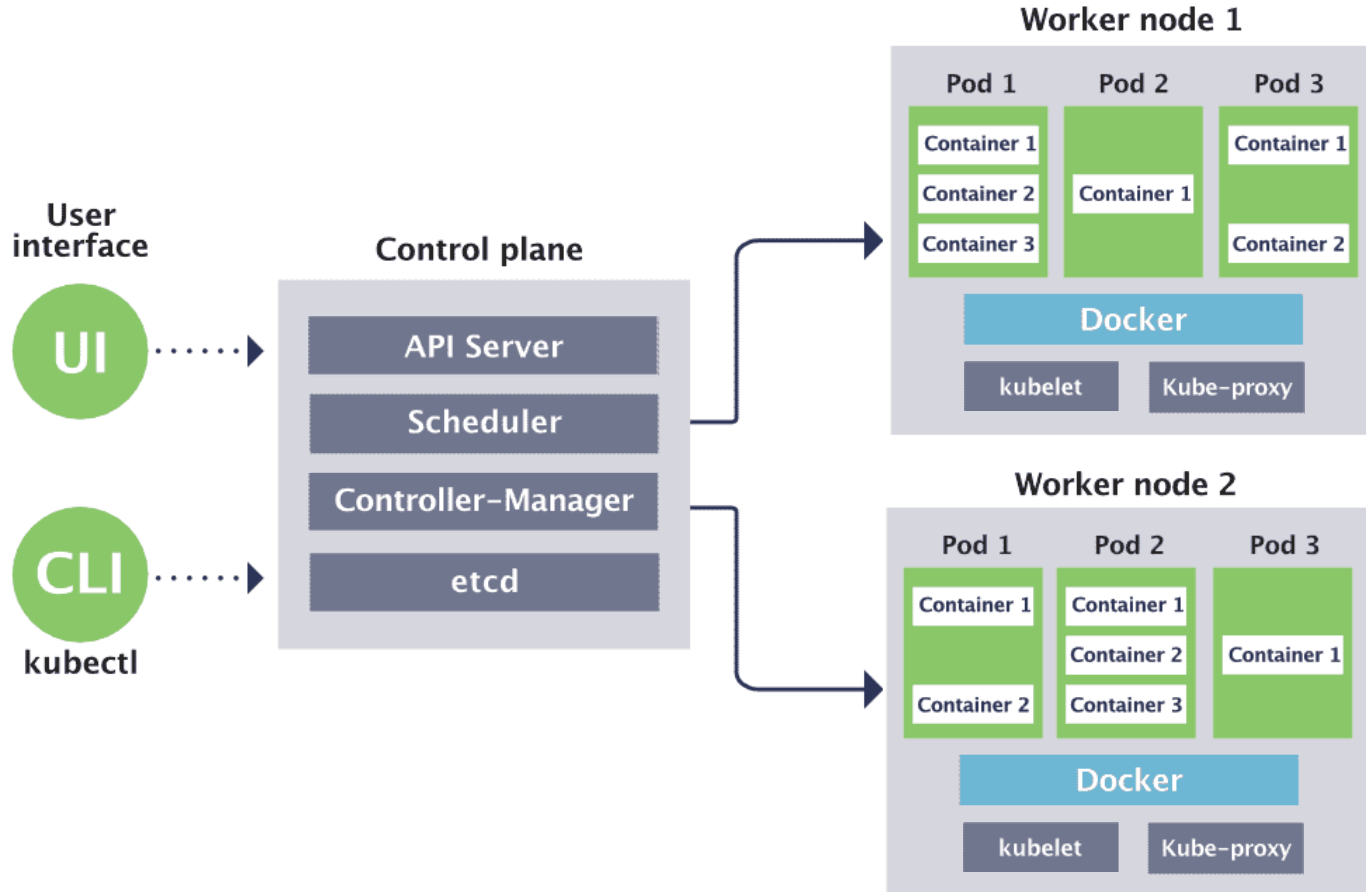
Docker kubernetes

- Kubernetes is a Greek word meaning "helmsman" or "pilot" of a ship
- It's a container orchestration service



kubernetes

Docker kubernetes

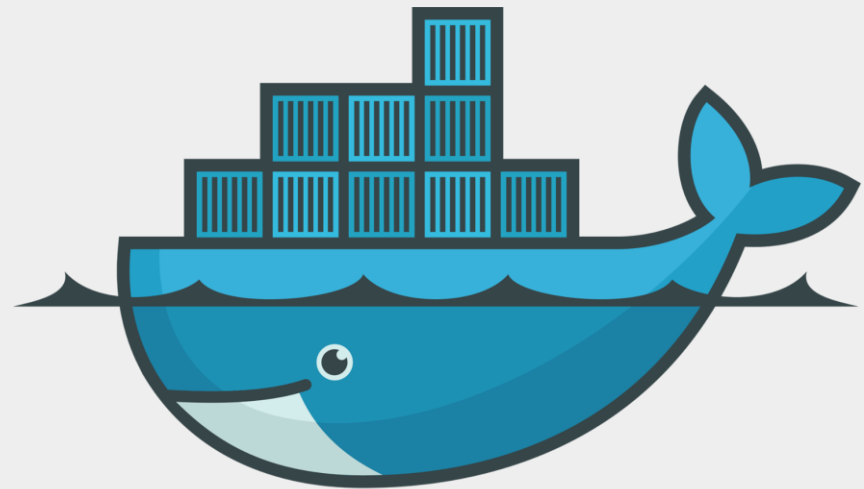


Docker kubernetes vs compose

- **Docker Compose** is a tool for running multi-container applications on a **single host**, while **Kubernetes (K8s)** is an orchestration platform designed to manage containerized workloads across a **cluster of many machines**.

Thank You!

Any Questions?



docker



LAB

Lab

Your task is as follows:

- Add logging to your code
- Containerize your application. (The app built in previous session)
- Show your logs
- You'll need to have those files:
 - Dockerfile
 - .dockerignore

Resources

<https://www.youtube.com/watch?v=PrusdhS2lmo>

<https://www.cncf.io/phippy/the-childrens-illustrated-guide-to-kubernetes/>